

Cyber Deception

Simone Iozzi

Contenuto

1

Stato dell'arte e ispirazioni industriali

2

Requisiti Funzionali e non Funzionali

3

Motore di IA - Isolation Forest

4

Stack tecnologico e Strumenti

5

Integrazione in WordPress

6

Casi d'uso

7

Metriche

Stato dell'arte e ispirazioni industriali

Stato dell'arte e ispirazioni industriali

Performance & Memory Safety (Pingora - Cloudflare):

- **Ispirazione:** Superamento dei limiti di NGINX in contesti ad alto traffico.
- **Uso:** i proxy non devono limitarsi a fare da “passacarte”, ma devono gestire la memoria e la concorrenza in maniera rigorosa e sicura. Si è scelto un linguaggio compilato (Go).

Extensibility & Observability (Envoy Proxy - Lyft/CNCF):

- **Ispirazione:** Filtri L7 dinamici e telemetria distribuita.
- **Uso:** Architettura modulare (filtri). Il traffico passa attraverso un interceptor che decide dinamicamente se inoltrare o inviare dati al servizio di analisi.

Cloud-Native & Go Ecosystem (Traefik):

- **Ispirazione:** Standardizzazione su Go per il networking moderno.
- **Uso:** Validazione di Go come linguaggio d'elezione per proxy cloud-native, grazie alle goroutine che garantiscono latenze costanti sotto stress.

Requisiti Funzionali e non Funzionali

Requisiti Funzionali

REQUISITO FUNZIONALE	DESCRIZIONE
INTERCETTAZIONE E ROUTING	Proxying trasparente del traffico HTTP/API con logica di smistamento dinamico.
CYBER DECEPTION(HONEY-URLs)	Iniezione dinamica di esche nel DOM (lato server) per identificazione deterministica dei bot.
ANALISI COMPORTAMENTALE IA	Estrazione feature a runtime e inferenza asincrona per Risk Scoring.
RISPOSTA ATTIVA (ACTIVE RESPONSE)	Deviazione immediata del traffico verso Honeypot + Tarpitting per esaurimento risorse attaccante.
DATA PIPELINE E ANTI-POISONING	Log strutturati (JSON) via OpenTelemetry; tag IsPoisoned=True per la sanitizzazione del dataset di addestramento.

Requisiti non Funzionali

REQUISITO NON FUNZIONALE	DESCRIZIONE
BASSA LATENZA	Overhead di routing e injection garantito inferiore a 2ms per il traffico legittimo.
ALTA CONCORRENZA	Gestione di migliaia di sessioni simultanee tramite architettura asincrona a goroutine.
ISOLAMENTO DI RETE	Segregazione logica dell'Honeypot; nessuna rotta di rete permessa verso il sistema reale.
DISPONIBILITÀ (FAIL-OPEN)	Protezione della business continuity: in caso di crash dell'IA, il traffico sano transita bypassando l'inferenza, quindi con protezione ridotta.
TRASPARENZA OPERATIVA	Framework Security by Design: nessuna modifica richiesta al codice sorgente del backend.

Motore di IA - Isolation Forest

Motore di IA - Isolation Forest

Approccio Non Supervisionato (Unsupervised Learning):

Il sistema non necessita di dataset etichettati (es. "questo è un hacker"). Impara direttamente a modellare la "normalità" del traffico, rendendolo capace di adattarsi autonomamente ai cambiamenti nel tempo.

Logica di Funzionamento:

L'algoritmo si basa sulla rarità delle anomalie. Poiché gli attacchi sono punti atipici e poco frequenti nel traffico web, vengono isolati molto più rapidamente (in pochi passaggi) rispetto ai dati di navigazione standard.

Performance per il Real-Time:

Rispetto a tecniche pesanti come le Reti Neurali (Autoencoder), l'Isolation Forest è computazionalmente leggero. Questo garantisce un'inferenza ultrarapida su CPU standard, permettendoci di rispettare il vincolo di latenza ($< 2\text{ms}$) del nostro proxy.

Efficacia contro gli Zero-Day:

A differenza dei sistemi tradizionali basati su signature, che bloccano solo attacchi noti, questo approccio identifica le minacce basandosi esclusivamente sulla deviazione dalla baseline di normalità. Questo lo rende estremamente efficace contro attacchi mai visti prima (Zero-Day).

Stack tecnologico e Strumenti

Stack tecnologico e Strumenti

CATEGORIA	STRUMENTO SCELTO	PERCHE'	ALTERNATIVA (SCARTATA)
EDGE / PROXY	GO(NET/HTTP)	Performance native vicine al C++, gestione asincrona semplificata tramite goroutine e libreria standard ultra-stabile per networking L7.	Rust, Node.js
IA / INFERENZA	FASTAPI + UVICORN	Architettura asincrona nativa che elimina i colli di bottiglia dei worker sincroni, massimizzando il throughput dell'inferenza ML.	Flask + Gunicorn
IA / ALGORITMO	ISOLATION FOREST	Isola le anomalie senza dataset etichettati, offrendo latenza < 2ms su CPU senza necessità di costose GPU.	Autoencoder, Reti Neurali
OBSERVABILITY	FLUENTBIT	Efficienza estrema (scritto in C) con footprint di memoria minimo, perfetto per l'integrazione in pipeline asincrone ad alte prestazioni.	Logstash, Fluentd
BIGDATA / STORAGE	GOOGLE BIGQUERY	Analisi SQL immediata sui log senza la complessità di gestione di cluster complessi	Elasticsearch
TESTING	K6 / VEGETA	Scenari dinamici in JavaScript eseguiti da un motore Go leggerissimo, facilmente versionabili su Git.	Apache JMeter, A. Bench

Integrazione in WordPress

Integrazione in WordPress

Proxy in Go (Il Core Online):

Gestisce il routing trasparente:
Richiesta → HTTP Listener → Security
Interceptor → Legit Traffic Router.

Zero parsing del DOM: L'iniezione
dell'esca è delegata al backend per
mantenere la latenza sotto i 2ms.

Integrazione WordPress (Offloading):

Sfruttiamo la Hook API (wp_footer)
per iniettare l'esca direttamente nel
DOM al momento del rendering della
pagina.

Il proxy in Go riceve quindi un flusso
di traffico "arricchito" senza dover
spendere cicli CPU per manipolare
l'HTML.

Workflow di Rilevamento (Caso di Studio):

1. Il plugin inietta file finti/Honey-URL non indicizzati nelle pagine WordPress.
2. Il bot (scraper/attaccante) cade nella trappola e tenta di accedere all'esca.
3. Il Security Interceptor del proxy rileva la rotta esca, identifica istantaneamente l'attaccante e dirotta il traffico verso l'Honeypot Applicativo.

Casi d'uso

Casi d'uso

Attore: Utente Umano (Browser).

Caso d'Uso 1: La Navigazione dell'Utente Legittimo

1. L'utente richiede una pagina.
2. Il Security Interceptor estrae le feature e interroga asincronamente l'AI Inference Client.
3. Il modello Python restituisce un Risk Score basso (es. 0.1).
4. Il Legit Traffic Router inoltra la richiesta a WordPress.
5. WordPress risponde con l'HTML.
6. L'URL Injector manipola l'HTML al volo, inserisce l'Honey-URL invisibile e lo manda all'utente.

Risultato: Navigazione fluida, nessun blocco, latenza mascherata.

Attore: Script automatizzato / Bot.

Caso d'Uso 2: Il Bot "Stupido" / Web Scraper (Innesco Trappola)

1. Il bot fa scraping della pagina legittima fornita nel Caso 1.
2. Leggendo il DOM, il bot trova il link nascosto (Honey-URL) e ci clicca sopra.
3. Il Security Interceptor riconosce la rotta esca e salta il controllo IA (è colpevole al 100%).
4. Instrada la connessione verso l'Honeypot Applicativo.
5. Il Telemetry Agent tagga immediatamente l'IP e la sessione come IsPoisoned=True.

Risultato: Il bot viene ingabbiato e l'IP marchiato.

Casi d'uso

Attore: Botnet avanzata / Attaccante umano che aggira le esche.

Caso d'Uso 3: L'Attaccante Sofisticato (Account Takeover)

1. L'attaccante punta dritto alla pagina di login (/wp-login.php) evitando le esche.
2. Il Security Interceptor estrae feature anomalie (es. header JA4, frequenza sospetta, assenza di header tipici).
3. L' AI Inference Client interroga Python. L'Isolation Forest nota lo scostamento dalla baseline e spara un Risk Score alto (es. 0.95).
4. Il proxy devia silenziosamente la richiesta sull' Honeypot Applicativo.
5. L'Honeypot fa Tarpitting (tiene la connessione aperta per 30 secondi) e poi restituisce un finto errore.

Risultato: L'attacco fallisce, le risorse dell'hacker vengono sprecate.

Attore: Sistema Schedulato (es. Job notturno).

Caso d'Uso 4: Pulizia e Riaddestramento (La Pipeline ML)

1. Lo script Python si collega a BigQuery.
2. Scarica i log dell'ultima settimana filtrando via tutte le righe con IsPoisoned=True (generate nei Casi 2 e 3).
3. Usa i dati puliti per riaddestrare la curva di normalità dell'Isolation Forest.
4. Aggiorna il modello in produzione nel container Servizio Inferenza IA.

Risultato: L'IA si evolve imparando il nuovo traffico sano, immune all'avvelenamento da parte degli hacker.

Metriche

Metriche

CATEGORIA	Metrica Chiave	Obiettivo / Descrizione
Performance	Latenza proxy	<2 ms di overhead per il routing di richieste lecite
Performance	Latenza Inferenza	<1 ms (Client Go → Container Python)
ML / Sicurezza	F1-Score / FP	Ottimizzazione per minimizzare i Falsi Positivi (FP ≈ 0)
Cyber Deception	Trap Trigger Rate	Rapporto tra tentativi su Honey-URL e richieste totali
Cyber Deception	Attacker Dwell Time	Tempo di permanenza del bot nell'Honeypot (massimizzare via Tarpitting)
Data Integrity	Anti-Poisoning	100% di corrispondenza tra record dell'Honeypot e log flaggati <i>IsPoisoned = True.</i>

**Grazie Per
l'Attenzione**